Technical Report

Solving a Custom Frozen Lake Environment Using Q-Learning

OPOKU ASANTE EMMANUEL

22426020

GitHub Repository

`https://github.com/Yawasante1/Yawasante1-RL-FrozenLake-Assignment`

June 17, 2026

1 Introduction

Reinforcement Learning (RL) is a machine learning paradigm in which an agent learns optimal behaviour through interaction with an environment. Unlike supervised learning, RL does not rely on labelled training data. Instead, the agent learns by receiving rewards or penalties for actions taken and gradually develops a policy that maximizes cumulative rewards.

Q-learning is one of the most widely used model-free reinforcement learning algorithms. It estimates the value of taking a particular action in a given state and learns an optimal policy without requiring prior knowledge of the environment. The objective of this project was to develop a Q-learning agent capable of solving a custom Frozen Lake environment. The agent was required to navigate from a start state to a goal state while avoiding holes that terminate the episode. Performance was evaluated based on the agent's ability to learn an optimal path and consistently reach the goal state.

2 Environment Design

A custom 8×8 Frozen Lake environment was implemented in Python. The environment contains four types of cells:

- S = Start State
- F = Frozen (Safe) State
- H = Hole (Terminal State)
- G = Goal State (Terminal State)

The environment layout is shown below:

S	F	F	F	F	F	F	F
F	F	F	F	F	F	F	F
F	F	F	H	F	F	F	F
F	F	F	H	F	F	F	F
F	F	F	H	F	F	F	F
F	H	H	F	F	F	H	F
F	H	F	F	H	F	H	F
F	F	F	H	F	F	F	G

The agent begins at the upper-left corner and must reach the goal located at the lower-right corner while avoiding holes.

Four actions were available to the agent.

Action Code	**Action**
0	Left
1	Down
2	Right
3	Up

Table 1: Available Actions

The reward structure used in the environment is presented in Table 2.

Event	Reward
Reach Goal	+10
Fall into Hole	-1
Normal Movement	-0.01

Table 2: Reward Structure

The positive goal reward encourages successful completion of the task, whereas the movement penalty encourages the agent to find shorter and more efficient paths.

3 Q-Learning Algorithm

The Q-learning algorithm learns the value of performing an action in a particular state through a Q-table. Each entry in the Q-table corresponds to a state-action pair and is updated iteratively during training.

The update equation is:

$$Q(s,a) = Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

where:

- $Q(s,a)$ = Current Q-value
- α = Learning rate
- γ = Discount factor
- r = Reward received
- s' = Next state

An epsilon-greedy exploration strategy was employed. During training, the agent selected a random action with probability ϵ and selected the best-known action with probability $(1 - \epsilon)$.

The hyperparameters used are shown below.

Parameter	Value
Learning Rate (α)	0.10
Discount Factor (γ)	0.99
Initial Epsilon	1.00
Minimum Epsilon	0.02
Epsilon Decay	0.99
Training Episodes	30,000
Maximum Steps	200

Table 3: Training Hyperparameters

4 Training Methodology

Training was conducted using a tabular Q-learning approach. At the beginning of each episode, the environment was reset to the start state. The agent then interacted with the environment using the epsilon-greedy action selection policy.

After every action, the environment returned the next state, reward and termination status. The Q-table was subsequently updated using the Q-learning update equation. Epsilon was gradually reduced during training so that exploration decreased while exploitation increased.

The agent was trained for 30,000 episodes. During training, the following metrics were recorded:

- Episode rewards
- Epsilon values
- Success rates
- Q-table values

The learned Q-table was saved as `qtable.npy`, while the extracted policy was stored in `policy.txt`.

5 Experimental Results

Training performance improved steadily throughout the learning process.

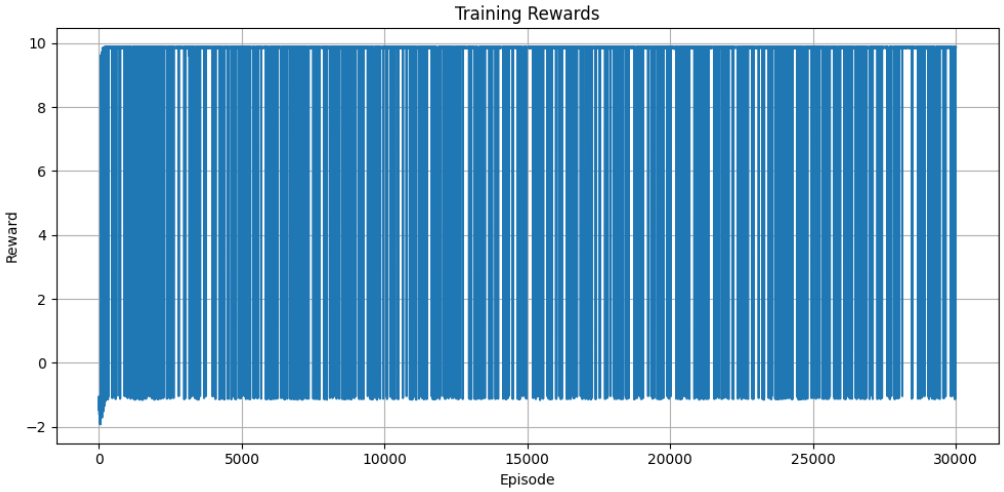


Figure 1: Training Rewards Across Episodes

Figure 1 shows the rewards obtained during training. The overall upward trend indicates that the agent gradually learned to reach the goal state more frequently while avoiding terminal hole states.

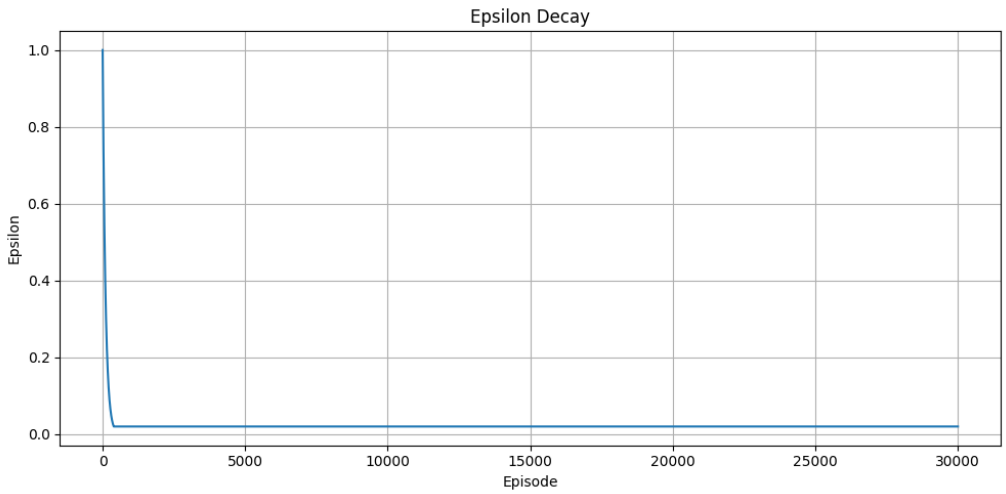


Figure 2: Epsilon Decay During Training

Figure 2 illustrates the reduction in exploration rate throughout training.

Episode	**Success Rate (%)**
1,000	81.50
2,000	90.05
3,000	92.87
5,000	94.94
10,000	96.78
20,000	97.67
30,000	97.92

Table 4: Training Progress

Final training results:

- Total Episodes: 30,000
- Successful Episodes: 29,377
- Training Success Rate: 97.92%

The maximum Q-value converged to 10.0, indicating successful propagation of the goal reward throughout the state space.

The trained policy was evaluated over 100 independent episodes.

Metric	**Value**
Success Rate	100.00%
Average Reward	9.87
Successful Runs	100
Failures	0

Table 5: Evaluation Results

The evaluation results demonstrate that the learned policy consistently reaches the goal state without failure.

6 Bonus Task: Exploration Strategy Comparison

Metric	Pure ϵ-Greedy	Decaying ϵ-Greedy
Training Success Rate	73.53%	97.92%
Successful Episodes	22,058	29,377
Final Epsilon	0.20	0.02
Evaluation Success Rate	Lower	100.00%

Table 6: Comparison of Exploration Strategies

To investigate the effect of exploration strategies, a comparison was conducted between a fixed -greedy strategy (= 0.20) and a decaying -greedy strategy. Both agents were trained for 30,000 episodes using the same environment and hyperparameters. The results in Table 6 show that the decaying -greedy strategy substantially outperformed the fixed strategy, achieving a training success rate of 97.92% compared to 73.53%. The improvement occurred because exploration was gradually reduced as learning progressed, allowing the agent to exploit the knowledge stored in the Q-table. Consequently, the decaying -greedy strategy was selected as the final approach for this project.

7 Learned Policy

The final policy extracted from the trained Q-table is:

R	R	R	R	D	D	D	D
R	R	R	R	D	D	D	D
U	R	U	H	R	D	D	D
U	R	U	H	D	R	R	D
D	R	U	H	U	R	R	D
D	H	H	U	U	U	H	D
L	H	R	L	H	D	H	D
L	R	D	H	L	U	R	G

The learned policy indicates that the agent successfully discovered a safe route from the start state to the goal state while avoiding terminal hole states. The policy primarily guides the agent toward the right and downward directions, which aligns with the location of the goal in the lower-right corner of the grid. The presence of directional changes near obstacle regions demonstrates that the agent learned to navigate around hazardous states rather than following a direct but unsafe path.

8 Challenges Encountered

Several challenges were encountered during implementation. Early versions of the environment and training scripts contained inconsistencies in attribute names, which caused errors during policy extraction.

A second challenge involved poor learning performance during initial experiments. The agent repeatedly achieved very low success rates due to inadequate reward shaping

and suboptimal hyperparameter values. This issue was resolved through reward engineering and careful tuning of epsilon decay parameters.

Another challenge involved file management. Generated result files were initially saved in an unexpected working directory because Python was executed from a different folder than the project location. This issue was resolved after identifying and correcting the working directory configuration.

9 Conclusion

This project successfully implemented a Q-learning agent capable of solving a custom 8×8 Frozen Lake environment. Through repeated interactions with the environment, the agent learned an effective navigation strategy that avoided hole states and consistently reached the goal state.

Using a decaying ϵ-greedy exploration strategy, the agent achieved a training success rate of 97.92% after 30,000 episodes, with 29,377 successful episodes recorded during training. Evaluation of the final policy over 100 independent test episodes produced a success rate of 100% and an average reward of 9.87, demonstrating stable convergence and reliable performance.

As part of the bonus task, a comparison was conducted between a fixed ϵ-greedy strategy and a decaying ϵ-greedy strategy. The fixed strategy achieved a training success rate of 73.53%, whereas the decaying strategy achieved 97.92%. The results showed that gradually reducing exploration while increasing exploitation significantly improved learning efficiency and policy quality. Consequently, the decaying ϵ-greedy strategy was selected as the final approach for this project.